Український Католицький Університет

ФАКУЛЬТЕТ ПРИКЛАДНИХ НАУК

Проектний Менеджмент в IT
Лекція 2

Ярослав Олексюк

2026-09-21

Документ:

YO-NK01-D102_02

ЗМІСТ

1 Продукт

2 Життєвий Цикл Продукту

3 Вимоги

1 Продукт

1. Визначення
2. Структура

1 1 Продукт. Визначення

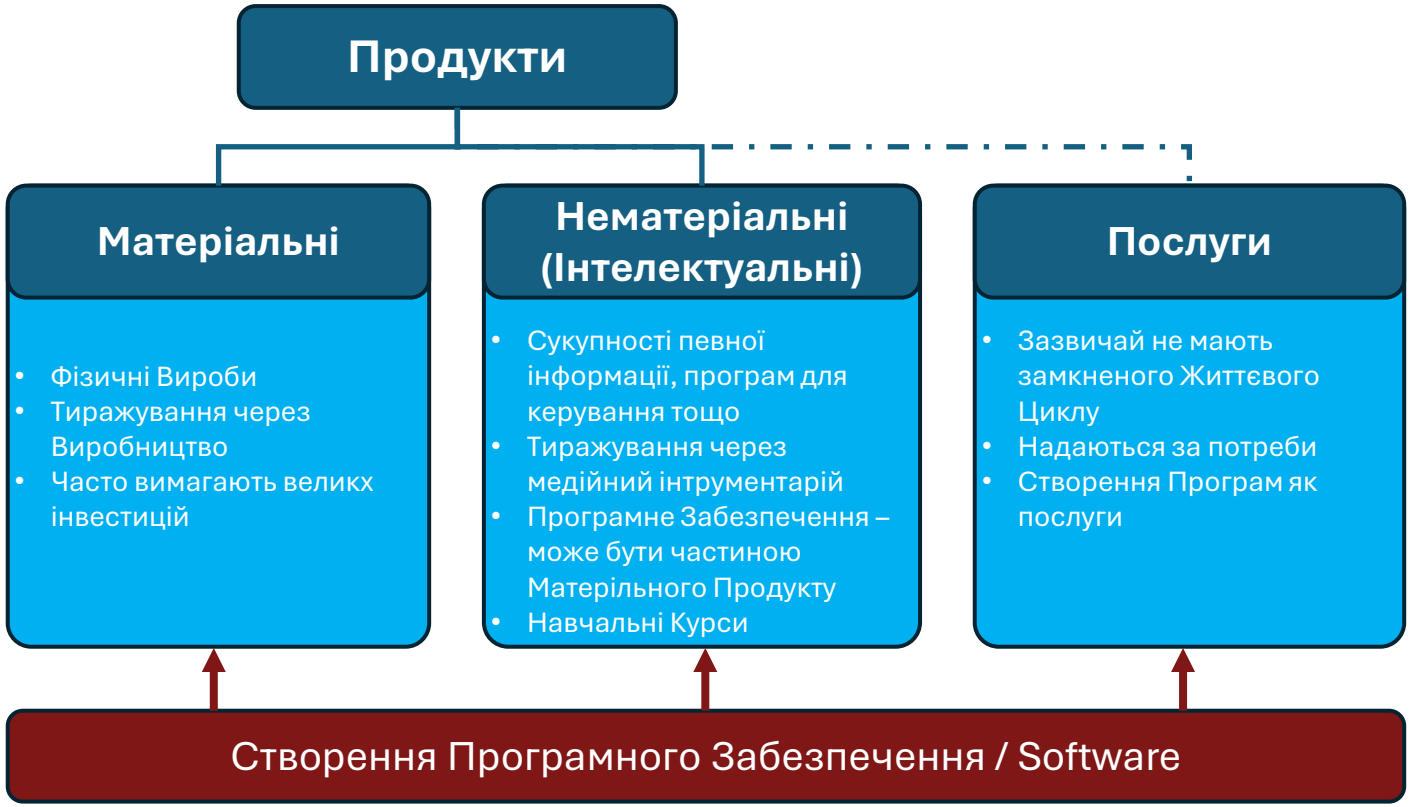


- Що таке Продукт у контексті Проекту ?
- Як можна класифікувати Продукти ?

1 1 1 Продукт. Визначення

Продукт / Product:

- ✓ **Об’єкт, для створення котрого виникає Проект**
- ✓ Мета і Результат виконання Проекту
- ✓ Матерільний Продукт зазвичай називається **Виробом**



1 1 2 Продукт. Апаратний та Програмний Продукти

Апаратні Продукти / Hardware Products та Програмні Продукти / Software Products:

- ✓ Відрізняються **рівнем інвестицій**, а отже **фінансовими ризиками** при **Створенні**
- ✓ **Програмні Продукти** зазвичай **можна оновити та виправити** порівняно низьким коштом у багатьох Клієнтів; **Ембедед Програми** низького рівня часто **не можливо виправити** у Клієнта без заміни Апаратної бази
- ✓ **Апаратні Продукти** практично **немає змоги виправляти**, тож підтримка здебільшого полягає у **Гарантії**, а також вирішенні проблем, у .т.ч через **повернення** виготовлених Виробів



1 2 Продукт. Структура

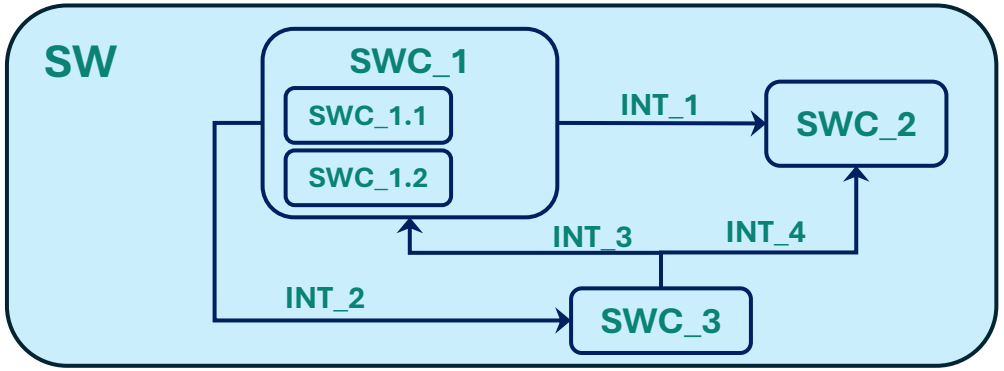


- З чого складаєтсья Виріб ?
- З чого складається Програмний чи Апаратний Продукт ?
- Як можна описати Структуру Продукту ?
- Для чого описувати Структуу Продукту ?

1 2 1 Продукт. Структура Програмного Продукту

Структура Програмного Продукту:

- ✓ **Архітектура** описує Програму як **стуктуру Компонентів та Інтервейсів** (зв'зків) між ними.
- ✓ **Архітектура / Architecture** – це:
  - ✓ ***Структура (структури) системи***, *що складається з* ***програмних елементів, зовнішньо помітних властивостей*** *цих елементів та* ***взаємозв'язків*** *між ними / The* ***structure(s) of a system****, comprising* ***software elements****, the* ***externally visible properties*** *of those elements, and the* ***relationships*** *among them* [*Software Architecture in Practice*, 2nd Edition, Bass, Clemens, Kazman Addison-Wesly, 2003]
  - ✓ ***Сукупність структур***, *необхідних для* ***аналізу системи***, *що включає* ***програмні елементи, взаємозв'язки*** *між ними та* ***властивості*** *обох / The* ***set of structures*** *needed to* ***reason about the system****, which comprises* ***software elements, relations*** *among them, and* ***properties*** *of both* [*Documenting Software Architectures: Views and Beyond*, 2nd Edition, Clemens et el., Addison- Wesley, 2010]

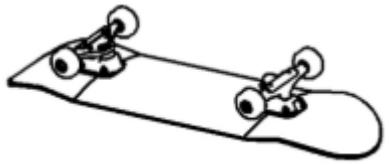


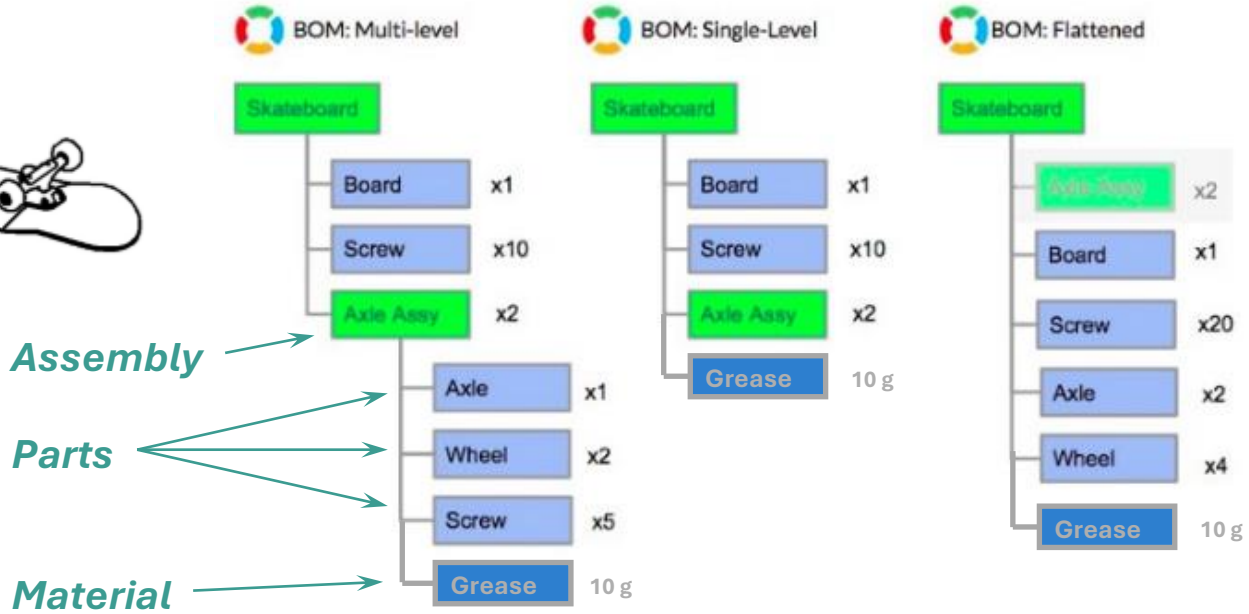
- ❖ **Архітектура – ключовий рівень** створення Програм, котрий **визначає фукнціонування** Програми та **подальший її розвиток** на ієрархічних рівнях.
- ❖ Необхідно намагатися **відразу будувати добру Архітектуру** (стабільну, ефективну, придатну до можифікацій та масштабування тощо), тому що пізніше її **зміна може бути дуже ускладненою** або навіть неможливою (недоцільною).

1 2 2 Продукт. Структура Апаратного Продукту чи Системи

Структура Виробу:

- ✓ Визначається **будовою** або/і **функціоналом**
- ✓ Зазвичай складаєтсья із Компонентів декількох стандартних типів:
  - ✓ **Складальні Одиниці (Assembly)** – сукупність Деталей та Матеріалів, котра є фізично одним об'єктом, функціонує як цілісність
  - ✓ **Деталі (Part)** – базові одиничні елементи, зазвичай з одного матеріалу, котрі виконують якусь одну функцію
  - ✓ **Матеріали (Material)** – незлічувані частини, котрі потрібні для функціонування
- ✓ Для Компонентів потрібно приймати важливе рішення: виготовляти самостійно чи купувати готове

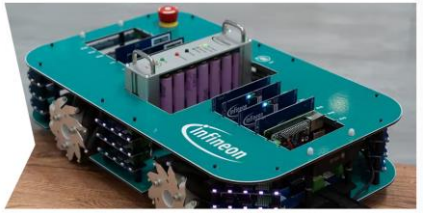




1 2 3 Продукт. Структура Апаратного Продукту чи Системи

Специфікація / Bill of Materials (BOM):

- ✓ Це інженерний документ, котрий
  - ✓ Формулює **Структуру Виробу** за його будовою чи/і функціоналом
  - ✓ Описує **склад Виробу**: **Компоненти**, їхня **кількість** в одному Виробі
- ✓ Застосовується як для **обладнання**, так і для **програмного забезпечення**



BOM (приклад)

	Project:			P101		Autonomous Demo Vehicle			
	Document:			P101-00101-01		Vehicle BOM			
	Ver., Date:			01		2026-03-11			
	Level			Part Number	Version	Title	Qty	Type	Origin
	I	II	III						
	0	**0**	**0**	**P101-00101**	**02**	**Vehicle**	**1**	**Asm**	**Make**
	1	**0**	**0**	**P101-01100**	**00**	**Chassis**	**1**	**Asm**	**Make**
	1	1	0	P101-01101	00	Frame	1	Asm	Make
	1	2	0	P101-01102	01	Wall	4	Asm	Make
	1	3	0	P101-01103	00	Cover	1	Asm	Make
	2	**0**	**0**	**P101-01200**	**01**	**Drive Module**	**4**	**Asm**	**Make**
	2	1	0	P101-12035	03	Hub	1	Asm	Make
	2	2	0	M00012345	A	Motor	1	Asm	Buy
	2	3	0	W00065432	B	Wheel	1	Asm	Buy
	2	3	1	P101-42123	00	Rim	2	Prt	Make
	2	3	2	R-001-200	AE	Roller	9	Prt	Buy
	2	3	3	S-M6-20	-	Screw M6x20	9	Prt	Buy
	2	3	4	S-M8-30	-	Screw M8x30	3	Prt	Buy
	3	**0**	**0**	**P101-01300**	**03**	**Energy Module**		**Asm**	**Make**
	3	1	0	B000567	C	Battery		Asm	Buy
	3	2	0	BCB-123-00-25	2.1.010	BMS Board		Asm	Buy
	4	**0**	**0**	**P101-01400**	**05**	**Control Module**		**Asm**	**Make**
	4	1	0	P101-31110	03	Main Control Unit		Asm	Make
	4	1	1	ABC8FD16GY	3.10.001	Main Ctrl Board		Prt	Buy
	4	1	2	P101-60010	1.00.020	Main Ctrl SW		Asm	Make
	4	2	0	P101-31120	01	Drive Control Unit		Asm	Make
	4	3	0	P101-31330	06	Connectors Assembly		Asm	Make

- ❖ **BOM** – основне джерело інформації про **Структуру Виробу**
- ❖ BOM повинен містити **Структуру Компонентів** з їхньою **ідентифікацією** (**Номер Деталі / Part Number**, **Версія / Version / Revision** тощо)

2 Життєвий Цикл Продукту

1. Визначення
2. Створення Продукту

2 1 Життєвий Цикл Продукту. Визначення

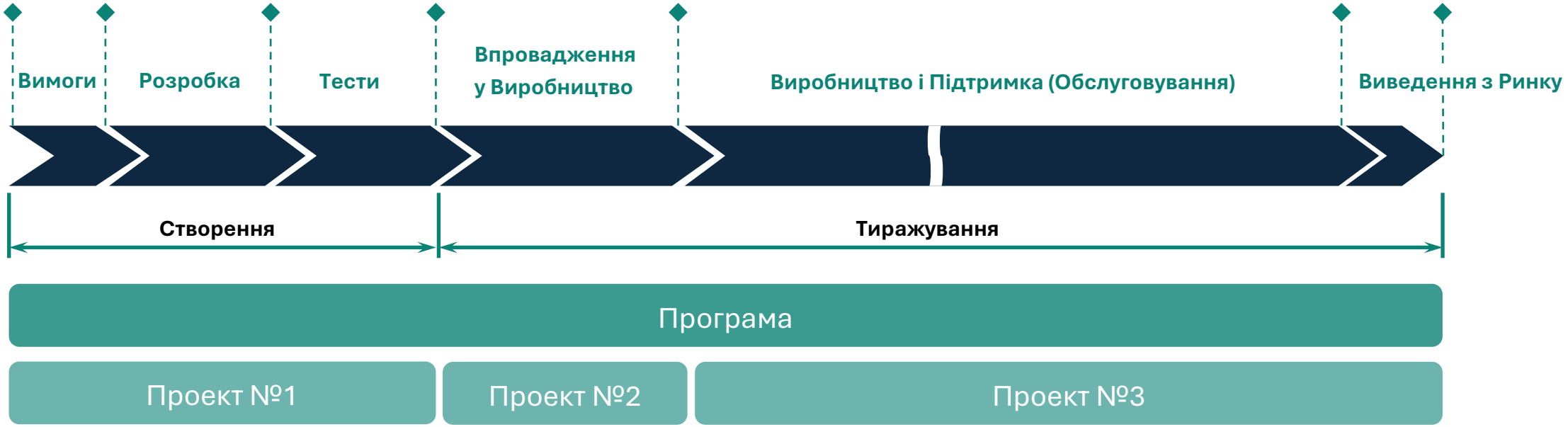


- Що таке Життєвий Цикл Продукту ?
- Які основні елементи Життєвого Циклу Продукту ?
- Чим відрізняється від Життєвого Циклу Проекту ?

2 1 Життєвий Цикл Продукту. Визначення

Життєвий Цикл Продукту:

- ✓ Проміжок часу **від ідеї до завершення** його **існування на ринку**
- ✓ Передбачає
  - ✓ **Створення Продукту** (Вимоги → Розробка → Тести)
  - ✓ **Тиражування Продукту** (Впровадження у Виробництво → Виробництво і Підтримка → Виведення з Ринку)



- ❖ Проект може охоплювати **цілий Життєвий Цикл Продукту або його частину**
- ❖ **Життєвий Цикл** поділений на **етапи**, котрі можуть бути **окремими Проектами** або частинами одного Проекту

2 2 Створення Продукту

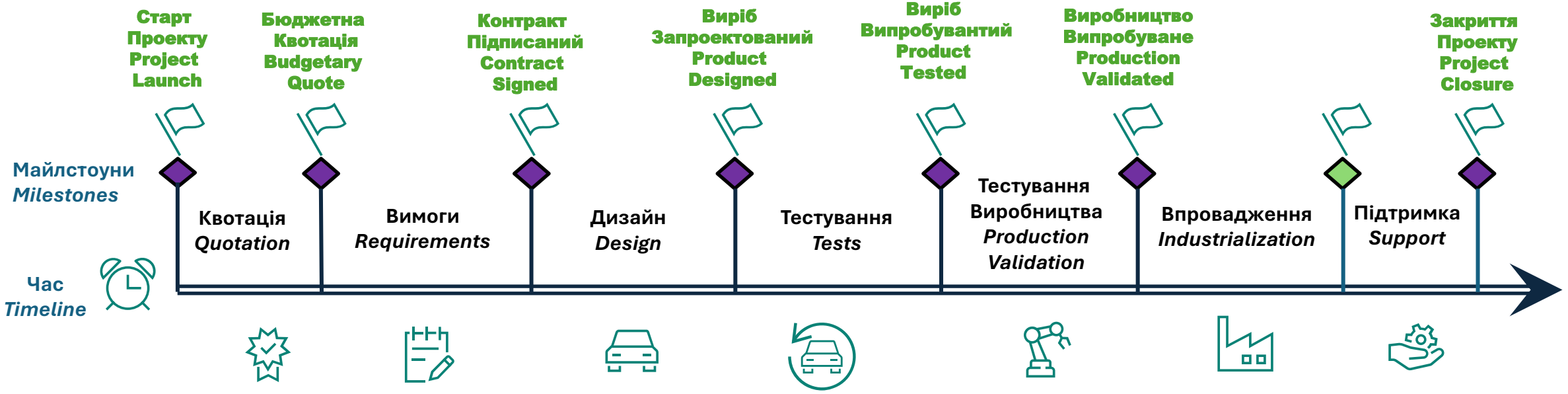


- Які підходи можна використати до Розробки Продукту ?
- Чим відрізняється Розробка Програмних і Апаратних Продуктів ?

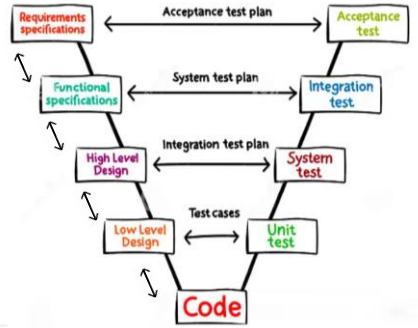
2 2 1 Створення Продукту. Послідовний Підхід

Послідовний (Sequential, Waterfall):

- ✓ **Обсяг Проекту / Project Scope** виконується послідоними діями до повної реалізації
- ✓ Проект планується за **Фазами / Phases**, між котрими визначають **Майлстоуни / Milestones**
- ✓ У кожній Фазі можливі **Ітерації** чи **Інкременти** у розробці, тестуванні тощо
- ✓ Проте після завершення Фази її **Обсяг** (Вимоги, Архітектура, Тести тощо) вважається **завершеним і незмінним**



2 2 2 Створення Продукту. V-Модель



Вимоги *Requirements* | Архітектура *Architecture* | Дизайн *Design* | Тестування *Tests*

System Level

Design Level (SW, HW)

Development

Decomposition

Testing

Integration

Простежуваність
Traceability

Vertical

Horizontal

bidirectional traceability
consistency

Stakeholder requirements

SYS.2 BP6
SYS.2 BP7

System requirements

SYS.5 BP5
SYS.5 BP6

System qualification test specification
Test cases

SYS.5 BP5

System qualification test results

SYS.3 BP6
SYS.3 BP7

System architecture

SYS.4 BP7
SYS.4 BP8

System integration test specification
Test cases

SYS.4 BP7

System integration test results

SWE.1 BP6
SWE.1 BP7

Software requirements

SWE.6 BP5
SWE.6 BP6

Software qualification test specification
Test cases

SWE.6 BP5

Software qualification test results

SWE.2 BP7
SWE.2 BP8

Software architecture

SWE.5.BP7
SWE.5 BP8

Software integration test specification
Test cases

SWE.5 BP7

Software integration test results

SWE.3.BP5
SWE.3 BP6

Software detailed design

SWE.4 BP5
SWE.4 BP6

Unit test specification

SWE.4 BP5

Unit test results

Software units

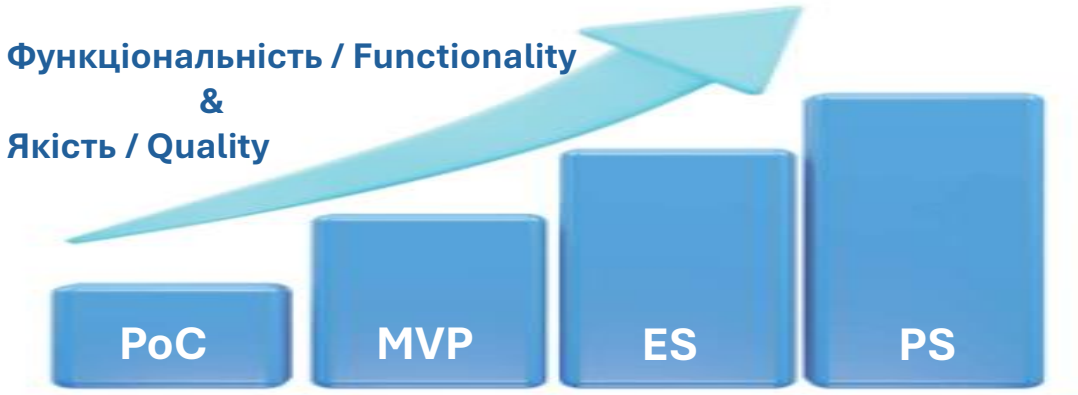
SWE.4 BP5

Static verification results

2 2 3 Створення Продукту. Інкрементний Підхід

Інкрементний / Incremental:

- ✓ **Функціональність Продукту** розділяється на **частини**, котрі реалізовуються **послідовно**
- ✓ Продукт випускається як **проміжні Релізи із частковою Функціональністю та нповними рівнями Якості**
- ✓ Кожний **наступний Реліз** зазвичай має **більшу Функціональність та вищою Якістю від попереднього**
- ✓ Приклади релізів:
  - ✓ **PoC (Proof of Concept)** – ранній варіант, для перевірки концепції, загального функціонування
  - ✓ **MVP (Minimal Viable Product)** – мінімально придатний Продукт, може мати всю функціональність
  - ✓ **Engineering Sample** – рання версія, котру тестують в межах розробки, не призначений для виробництва
  - ✓ **Production Sample** – виробничичй зразок, повністю фукнціональний та протестований
- ✓ Добре узгоджується із **Гнучким / Agile** та **Послідовним / Waterfall** способами організації Проекту

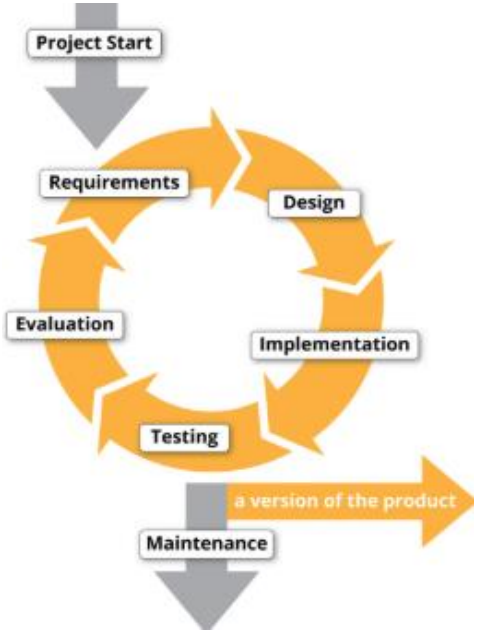


❖ Інкрементальний Підхід часто ефктивно застосовується у великих Програмах, де необхідно узгоджувати окремі Продукти, котрі створюються ізрізною шивдкістю

2 2 4 Створення Продукту. Ітеративний Підхід

Ітеративний / Iterative:

- ✓ Продукт **регулярно випускається із покращеннями** у циклах розробки – **Ітераціях**
- ✓ Нові релізи отримуютьнові **нові функції та/або покращення** наявних, а також виправлення помилок
- ✓ Передбачає, що **нова Ітерація** може ґрунтуватися на **нових Вимогах**, змінених порівняно з попередньою
- ✓ Добре узгоджується із **Гнучким / Agile** способом організації Проекту





❗
- ❖ Найчастіше немає змоги випускати Апаратний Продукт / Hardware Ітераціями, адже він мають один Початок Виробництва / Start of Production (SOP)
- ❖ Проміжні варіанти для тестування розглядаються все ж як Прототипи

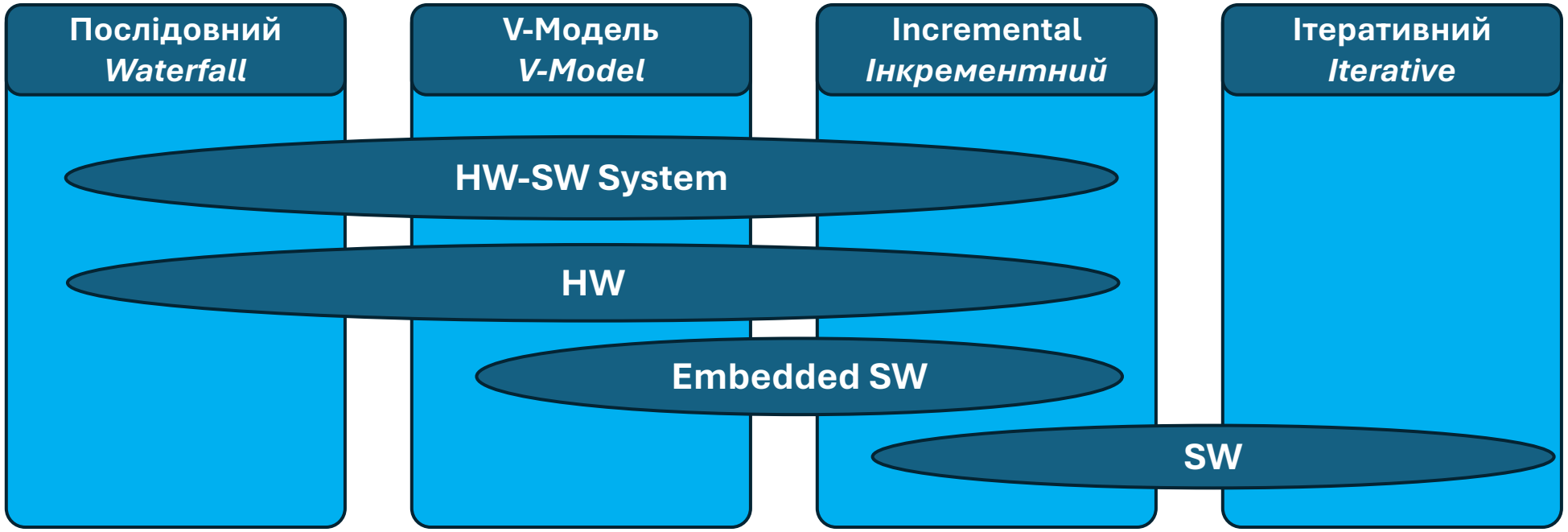
❗
- ❖ Варто **запам'ятати PDCA** як абревіатуру, діаграму, та головне – зміст PDCA зустрічається у багатьох методиках, підходах

2 2 5 Створення Продукту. Вибір Підходу

Застосування Підходів:

- ✓ Для **Апаратних** та **Програмних Продуктів** доцільними є **різні Підходи**, так само як і для **Апаратно-Програмних Систем / HW-SW Systems**

Розподіл Продуктів за Підходами (Приклад)



❖ Хоч Послідовний Підхід вважається «класичним», а Ітеративний чи Інкреметний «новітніми», ідеї всіх описаних Підходів застосовуються напрактиці вже давно (часто неявно та в інших формах): циклічне створення, додавання фукнціональності чи рівня якості етапами, рівні розробки та відповідні рівні тестування; проте останніми десятиліттями вони виділяються в окремі цілісні моделі.

3 Вимоги

1. Визначення
2. Ієрархія
3. Класифікація
4. Атрибути
5. Формат
6. Опрацювання
7. Документцвання

3 1 Вимоги. Визначення



- Що таке Вимоги ?
- Які харатеристики Вимог ?
- Що є Вимогами, а що ними не є ?
- Що таке SMART ?

3 1 1 Вимоги. Визначення

«***Вимога*** *— це: 1.* ***Потреба****, яку усвідомлює зацікавлена сторона. 2.* ***Можливість*** *або* ***властивість****, якою* ***повинна володіти система****. 3.* ***Документальне відображення*** *потреби, можливості або властивості..*»

“**Requirements** is: 1. *A* ***need*** *perceived by a stakeholder. 2. A* ***capability*** *or* ***property*** *that a* ***system shall have****. 3. A* ***documented representation*** *of a need, capability. or property.”*

International Requirements Engineering Board (IREB)

«**Опис Потреби** *— це результат формального перетворення одного або кількох джерел чи концепцій життєвого циклу на* ***узгоджене очікування*** *щодо того, що* ***об’єкт виконуватиме певну функцію*** *або* ***матиме певну якість*** *у межах визначених* ***обмежень*** *із* ***прийнятним рівнем ризику****.*»

"*A* ***Need Statement*** *is the result of a formal transformation of one or more sources or lifecycle concepts into an* ***agreed-to expectation*** *for an entity to* ***perform some function*** *or* ***possess some quality*** *within specified* ***constraints*** *with* ***acceptable risk.***"

International Council on Systems Engineering (INCOSE)

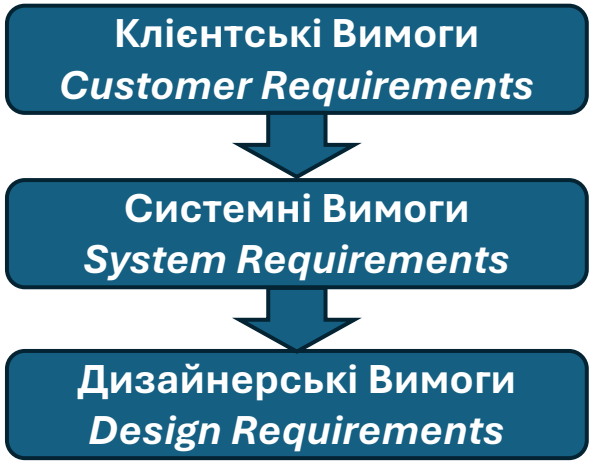
«***Формулювання Вимоги*** *— це результат формального перетворення одного або кількох джерел, потреб або* ***вимог вищого рівня*** *в* ***узгоджене зобов’язання*** *суб’єкта* ***виконувати певну функцію*** *або* ***володіти певною якістю*** *в межах визначених* ***обмежень*** *із* ***прийнятним рівнем ризику****.*»

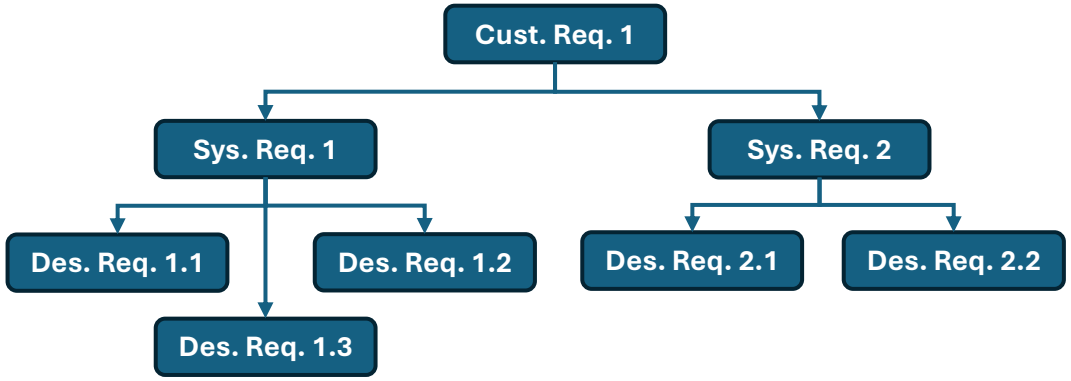
"*A* ***Requirement Statement*** *is the result of a formal transformation of one or more sources, needs, or* ***higher-level requirements*** *into an* ***agreed-to obligation*** *for an entity to* ***perform some function*** *or* ***possess some quality*** *within specified constraints with* ***acceptable risk****.*"

International Council on Systems Engineering (INCOSE)

3 2 Вимоги. Ієрархія

- ✓ У загальному говорять про **Вимоги / Requirements до певної Системи**, де **Система** – це наш Виріб / Продукт
- ✓ У дещо точнішому описі часто розділять 3 основні рівні, для котрих формулюють Вимоги:
  - ✓ **Клієнтські Вимоги / Customer Requirements**
    - ✓ Найвищий рівень Вимог
    - ✓ Описують очікувану поведінку Системи для Користувача
    - ✓ Можуть не описути технічних деталей, а лише користувацькі потреби
  - ✓ **Системні Вимоги / System Requirements**
    - ✓ Описують Систему на високому рівні, де елементи Системи – це неподільні компоненти (black box)
  - ✓ **Дизайнерські Вимоги / Design Requirements**
    - ✓ Описують елементи Системи у всіх необхідних деталях (white box)
- ✓ Вимоги кожного нижчого рівня є **декомпозицією / уточненням** Вимог вищиго рівня





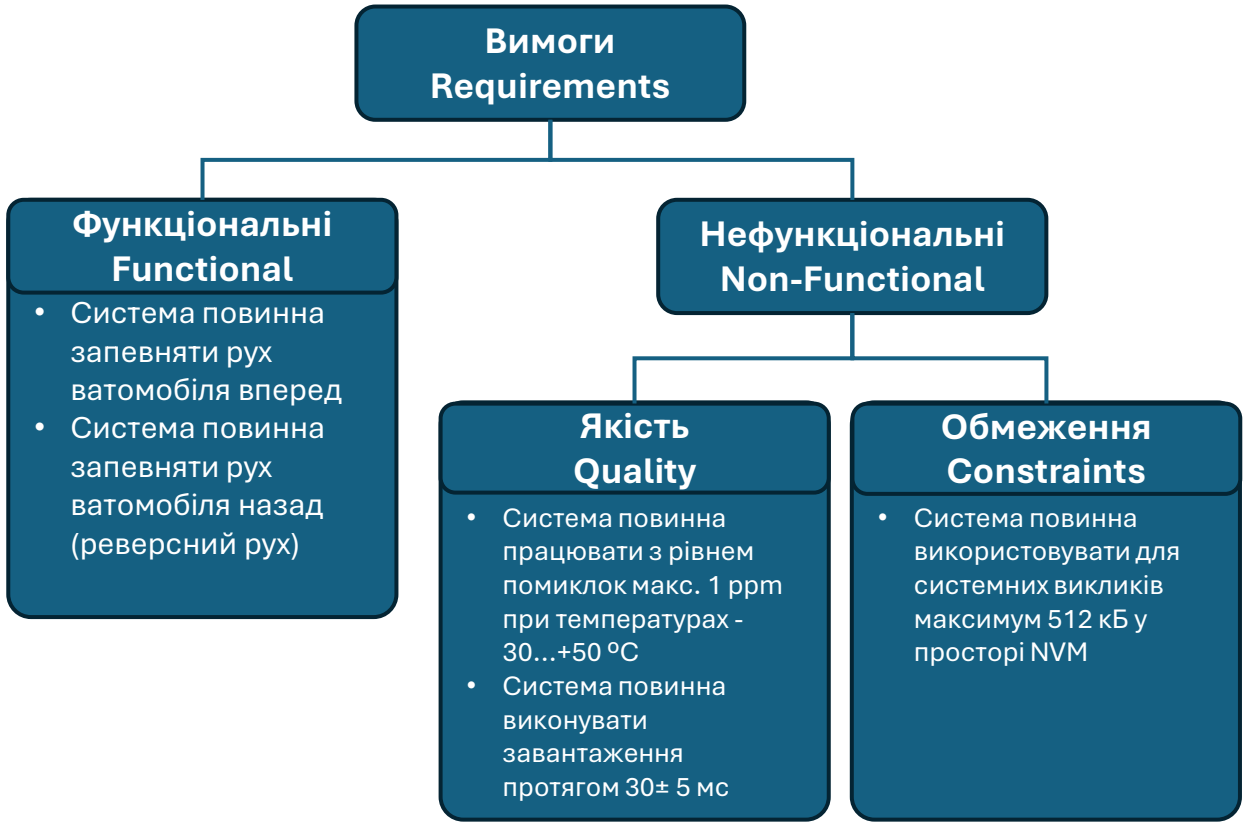
3 3 Вимоги. Класифікація



- Як можна класифікувати Вимоги ?
- Які можуть бути джерела Вимог ?

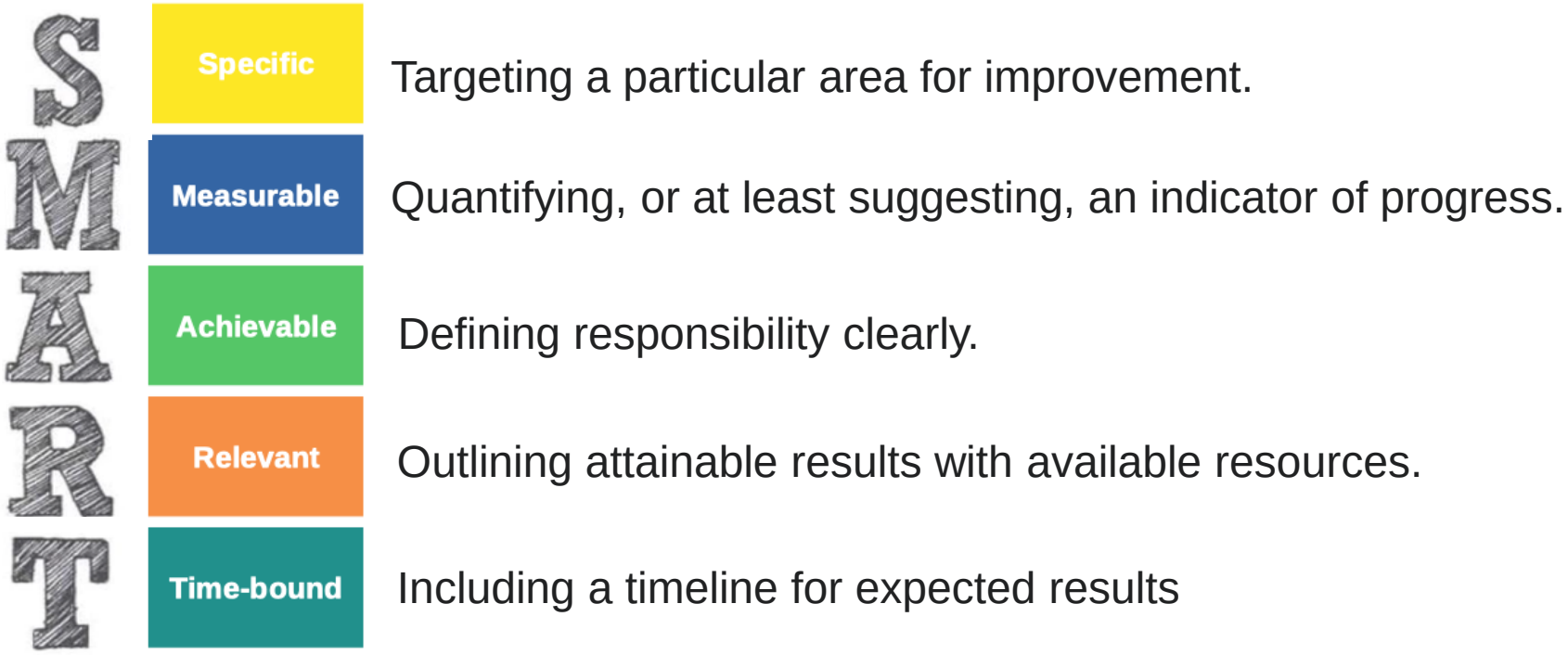
3 3 1 Вимоги. Класифікація

- ✓ **Вимоги / Requirements** найчастіше класифіковують за такими параметрами:
  - ✓ **Функціональні / Functional –**
    - ✓ Що описувана Система повинна виконувати, її фнукціонал
  - ✓ **Нефукнціональні / Non-Functional**
    - ✓ Як добре Система повинна працювати,
    - ✓ Атрибути та характеристики Якості та Обмеження



3 4 Вимоги. Атрибути

- ✓ **Вимоги / Requirements повинні бути** чіткими, вимірюваними, досяжними, відповдіними, обмеженими в часі
- ✓ Вимоги Клієнтів часто не відповідають цим характеристикам, тому потрібно їх уточнювати
- ✓ Також важливою **атомарність**: окрема Вимога повинна говорити про якусь одну Потребу
- ✓ Модель **SMART** добре описує головні характеристики Вимог



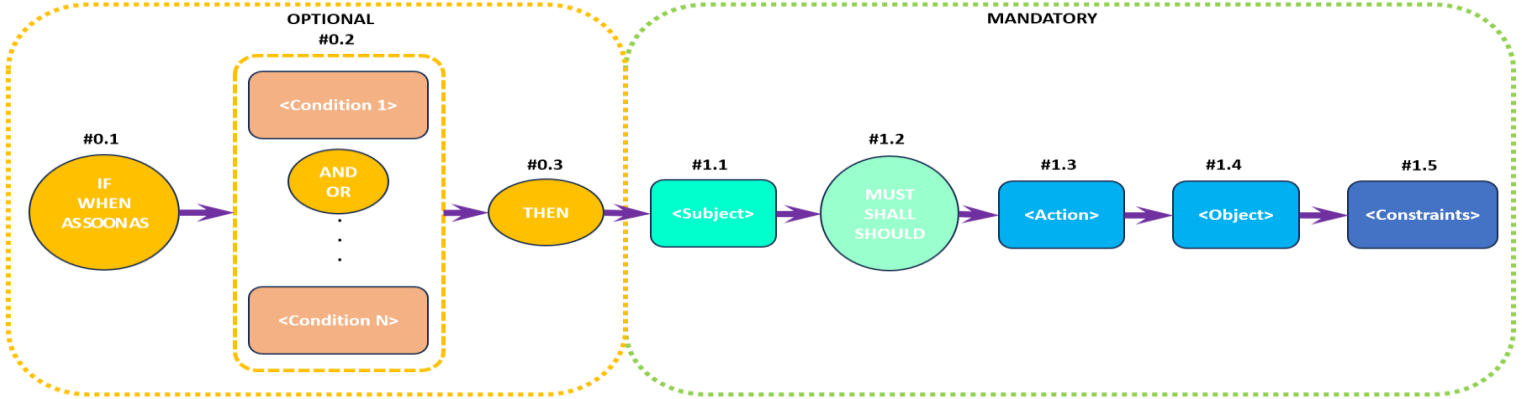
- ❖ **Якщо Вимогу не можна виміряти чи проконтролювати, - це не Вимога**
- ❖ Вимоги повинні бути чіткими та однозначними, щоб отримати чіткі та однозначні результати

35 Вимоги. Формат

✓ Вимоги зазвичай повинні бути написані в **узгодженому стандартному форматі (Notation)**.

Стандартний формат написання Вимог (INCOSE):

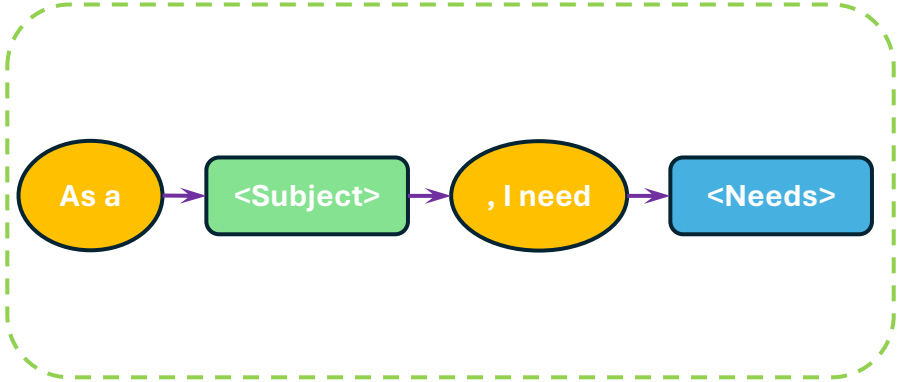
- Формалізований підхід
- Описується Система та що вона має виконати, якими характеристиками володіти, за яких умов та обмежень
- Застосовуєтья у складних Системах



Item		
#	**Title**	**Description**
0.	**Pre-Condition**	**Condition under which Requirement is applicable.**
0.1	Conjunction	Entering clause which introduces condition type. Following conjunctions keywords are presumed: If - general condition; When - time condition; As soon as - trigger event condition.
0.2	Condition	Content of Condition for Requirement. There can be several Conditions split with logical operators: And; Or.
0.3	Adverb	Linkage key word between Condition and Requirement contents. Keyword: Then.
1.	**Requirement**	**Core Content of Requirement.**
1.1	Subject	Subject which implements the Requirement. Following Subjects can be used: Product; System; Component; Function.
1.2	Modal Verb	Modal verb which determines obligation to follow the Requirement. Following keywords to be used: Must - mandatory, high level of importance; Shall - general mandatory; Should - conditional, with the condition to be described.
1.3	Action	Qualitative definition of Requirement which states what exactly is required. It has verbal form with active voice definition of an action Actor should do.
1.4	Object	Object on which Requirement to be applied.
1.5	Constraints	Quantitative definition of Requirement which provides precise boundaries for the Requirement. Follow boundary domains can be defined: Time; Physical values; Context conditions; Operational modes; Use cases.

Формат Вимог з точки зору Користувача:

- Розповідний підхід
- Описуються очікування / потреби з точки зору Користувача, часто від першої особи
- Застосовується в IT, до користувацьких програм



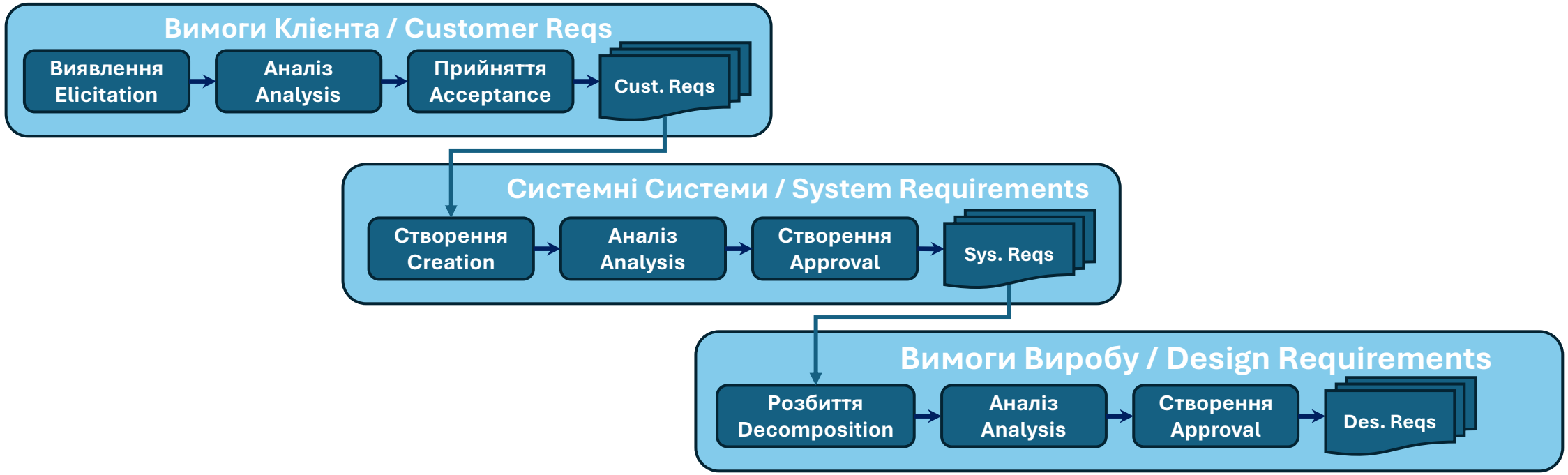
2 5 Вимоги. Опрацювання



- Які дії потрібно виконати з Вимогами ?
- У чому має полягати Опрацювання Вимог ?

3 6 Вимоги. Опрацювання

- ✓ Вимоги від Клієнта чи вищого рівня потрібно поарцювати:
  - ✓ Виявити: зібрати усі Потреби та Вимоги
  - ✓ Аналізувати: перевірити правильність формулювання, можливість реалізації
  - ✓ Прийняття: вести переговори, прийняти чи відхилити Вимоги котрі можливо чи не можливо реалізувати
- ✓ Системні Вимоги зазвичай створюють у межах Проекту на основі Вимог Клієнта
- ✓ Дизайнерські Вимоги – це зазвичай розбиття Системних на точніші елемети, зокрема із пропозиціями імплментації



- ❖ **Клієнт не завжи має рацію**: **Вимоги Клєінта** потрібно **аналізувати** із адекватним рівнем критики, адже Клєінт може не знати всіх можливих **нюансів**, **ризиків** чи **обмежень** реалізації Вимог
- ❖ **Відповідальність Проектної Команди** – це **приняти чи відхилити** Вимоги відповідно до можливості їх реалізації у контексті Проекту

37 Вимоги. Документування.

- ✓ **Від Клієнта** Вимоги можуть прийти у різних форматах
  - ✓ Через **погоджену систему обміну даними**
  - ✓ **Файлами** надісланими на пошту тощо – це можуть бути файли **Excel**, **Word** тощо
- ✓ У межах **Організації** найчастіше існує **визначена система та формат** документування Вимог
  - ✓ **Jira** – Вимоги формулюються як **тікети**; застосоувається в IT
  - ✓ **Confluence – сторінки** із описами Вимог, Потреб тощо; застосовується в IT
  - ✓ **Polarion, Jama, DOORS** - **інтерактивні документи** із Вимогами у стандртизованому форматі, котрі **пов'язані** між собою (рзні рівні Вимог), а також з Архітектурою, Дизайном, Тестами; застосовується у промисловості, зокрема в Автомотіві

Excel, Word	Jira	Confluence	Polarion, Jama, DOORS
• Вимоги від Клієнта • Найчастіше переносяться у стандртне середовище Організації: Jira, Confluence, Polarion тощо	• Лаконічний виклад • Можуть бути не дотримані атомарність та інші формальні атрибути • Поширене в IT	• Можуть бути не дотримані атомарність та інші формальні атрибути • Поширене в IT	• Стандратний формат Вимоги • Зв'язки з іншими Виомгами, Архітектурою, Дизайном, Тестами • Промивловий стандарт

❖ Чим складніша Система, Продукт, тим більше Вимог, а отже і тим складніше середовище потрібне для їх опрацювання

Проектний Менеджмент в IT
Лекція 2

Дякую за увагу ☺